

KUPC Project Progress Report

Kathmandu University Placement Connect — Backend & Frontend Status

Report date: July 17, 2026

This report compares completed work against the KUPC Backend Specification v2.0 roadmap, internal phase documentation, and the current state of the frontend prototype. The frontend was reviewed at `c:/projects/KU-Placement-Cell/frontend`.

Executive Summary

The backend has completed Phases 1–4: project setup, authentication, database implementation, and the full resume upload + AI analysis pipeline (62 automated tests passing for Phase 4). The frontend is a high-fidelity Figma Make prototype with rich mock UI across student, company, and admin flows, but it has **zero backend integration** — no API client, no auth screens, and all data is hardcoded in `App.tsx`.

Per Backend Specification v2.0, the next listed feature phase is **Job Posting & Discovery**. However, Student & Company Profile APIs (v2 Phase 3) were deferred during implementation and remain unbuilt. Profiles and frontend integration are prerequisites before job posting can deliver end-to-end value.

Backend — Completed Work

Phase	Scope	Status	Evidence
1	Project setup & architecture	Complete	Express + TS scaffold, env validation (Zod), logging, CORS, helmet, rate limiting
2	Authentication & authorization	Complete	Student/company/admin auth, JWT + refresh rotation, RBAC, OTP, 50+ auth tests
3A	Database design	Complete	Schema design doc, ERD, conventions, RLS strategy
3B	Database implementation	Complete	Supabase tables, indexes, RLS, seed data (200 jobs), repository layer, tests
4	Resume upload & AI analysis	Complete	Storage upload, BullMQ worker, PDF extract, OpenAI scoring, read APIs, Swagger

Backend — APIs Currently Live

Module	Endpoints	Notes
Auth	POST register/student, register/company, login, refresh, logout; GET /me	Production ready
Resumes	POST /resumes (upload), GET list, GET by id, GET analysis, DELETE	Asynchronous processing
Jobs	POST /jobs (placeholder only)	Returns stub response; repository exists but no service layer
Dashboard stubs	GET student/admin/company dashboard	RBAC smoke routes only

Backend — Remaining Work (per v2 spec)

v2 Phase	Feature	Current State
3	Student & Company Profiles	DB + repositories only; no profile CRUD APIs or storage uploads
5	Job Posting & Discovery	Placeholder route; jobs.repository ready; no search/filter/saved-jobs APIs

v2 Phase	Feature	Current State
6	Swipe Engine	swipes.repository exists; no API or business logic
7	Matching Engine	matches + conversations repos exist; not wired
8	Real-Time Chat	messages repo exists; no WebSocket/SSE layer
9	Notifications	notifications repo exists; no delivery service
10	Search & Recommendations	Not started (pgvector, embeddings)
11	Company Verification Workflow	Admin UI in frontend mock only; partial DB support
12–18	Dashboards, analytics, admin, security, docs, etc.	Mostly not implemented

Frontend — Completed Work

Area	Status	Details
Tech stack	Complete	React 18, Vite 6, Tailwind v4, motion, recharts, MUI icons
Landing page	Complete	Role entry (student / company / admin) with branded hero
Student UI	Mock complete	Dashboard, discover (swipe), matches, chat, resume analyzer, profile, settings, notifications
Company UI	Mock complete	Dashboard, job posting form, applicant kanban, discover students, analytics placeholder
Admin UI	Mock complete	Overview, company approval queue, user management, analytics charts
Design system	Partial	47 shadcn/ui components in components/ui/ — largely unused; custom inline components used
Product spec	Complete	kupc-figma-prompt.md defines full screen inventory and brand guidelines

Frontend — Remaining Work

- **Backend integration (0%)** — No fetch/axios calls, no .env, no API base URL configuration.
- **Authentication UI** — No login, register, OTP verification, or session/token management screens.
- **Routing** — Single monolithic App.tsx (~2,100 lines) with useState screen switching; no react-router.
- **Resume analyzer** — Simulated with setTimeout; not connected to POST /api/v1/resumes or analysis polling.
- **Job posting** — Form UI exists but has no submit handler or API integration.
- **Company profile editor** — Placeholder screen only ('Update your banner, about...').
- **Discover / swipe / chat / matches** — Fully mocked with hardcoded COMPANIES, STUDENTS, MESSAGES arrays.
- **Apply-to-job flow** — Not implemented.
- **State persistence** — No localStorage/session handling; refresh loses all state.
- **Error/loading states** — Not wired for real network calls.

Alignment Gap Analysis

Internal documentation renumbered phases for pragmatic delivery: database work became Phase 3, resumes became Phase 4. Phase 4 documentation explicitly states the next backend milestone is **Student & Company Profiles**, while Backend Specification v2.0 lists that as Phase 3 (not built as APIs) and positions **Job Posting & Discovery** as Phase 5.

Capability	Backend	Frontend	Gap
Auth	Complete	Missing UI	Users cannot log in from the app
Student profile	DB only	Mock static data	No read/update flow
Company profile	DB only	Placeholder screen	No CRUD or logo upload
Resume AI	Complete pipeline	Fake 2s timeout	Highest-value quick win once wired
Jobs	Placeholder API	Form UI, no handler	Neither side is production-ready
Swipe/Match/Chat	DB repos only	Rich mock UI	Large integration effort ahead

Test & Quality Snapshot

Suite	Command	Status
Phase 2 auth	npm run test (auth suites)	Passing
Phase 3 DB/repos	npm run test:phase3	Passing
Phase 4 resumes	npm run test:phase4	62 tests, 11 suites — all pass
Typecheck	npm run typecheck	Passing
Frontend tests	—	None configured
E2E tests	—	Not started

Recommended Next Phase

Phase 5 — Student & Company Profiles + Frontend Integration Foundation (see companion specification PDF). Job Posting & Discovery should follow as Phase 6, matching Backend Specification v2.0 ordering once profiles, auth UI, and the API client exist.

Rationale: the resume pipeline (Phase 4) and auth layer (Phase 2) are production-ready on the backend but unreachable from the frontend. Company job posting requires verified company context and profile data. Building job APIs before profiles would produce endpoints the UI still cannot consume meaningfully.